

Frontend Developer — Detailed Guide

Retail Demand Forecasting (Walmart M5, Hierarchical) · Team Study Material

1. What the frontend is

The **frontend** is everything that runs in the user's browser: the pages, the charts, the buttons. It holds no data of its own — it asks the backend for numbers and draws them.

```
BROWSER                                SERVER
React app  ---- GET /api/v1/... ---->  FastAPI
              <--- JSON -----
draw chart
```

Frontend vs backend. Frontend = what you see and click, running on the user's machine. Backend = the program holding the data, running on a server. They talk over HTTP.

Our stack, and why each piece is there:

Piece	What it is	Why we use it
React	A JavaScript library for building interfaces out of reusable pieces	The app has many screens sharing the same building blocks
TypeScript	JavaScript plus type checking	A backend field rename becomes a compile error instead of a blank chart
Vite	The build tool and dev server	Fast rebuilds; splits the output into chunks
Tailwind	Styling via utility classes	One palette defined in one config file; nothing drifts
Recharts	A React charting library	The one chart shape we need — history flowing into forecast
TanStack Query	Manages fetching, caching, loading and error state	The model is frozen, so most responses never change
React Router	Maps URLs to pages	Deep links like /forecast must work

2. React concepts, from zero

Component. A reusable piece of interface, written as a function that returns markup. Our `Badge`, `Card` and `ForecastChart` are components.

Props. Inputs passed into a component, like arguments to a function. `<Badge variant="good">API ready</Badge>` passes `variant` as a prop.

State. Data a component remembers between redraws. On the Assistant page the list of questions and answers is state — when it changes, React redraws.

Hook. A function starting with `use...` that lets a component use React features. `useState` holds state; `useEffect` runs a side effect. We also have our own: `useGenAIService`, `useSeriesDetail`.

Rendering. React works out what changed and updates only that part of the page — not the whole thing.

JSX. The HTML-looking syntax inside JavaScript. `<Card title="x" />` is not HTML; it is a component call written to look familiar. **SPA — Single Page Application.** The browser loads one HTML page and JavaScript swaps the content as you navigate, instead of fetching a new page from the server each time. Faster, but it means the server must be told to return the app for *any* URL — see Section 9.

3. Our pages

VERIFIED: 9 pages, in 01_PROJECT/frontend/src/pages/.

Route	File	Purpose
/	Overview.tsx	The 30-second answer: what is forecast, how well, on what data
/forecast	Forecast.tsx	Search a store-item; history flowing into the 28-day forecast, plus a planning view
/hierarchy	Hierarchy.tsx	Drill from chain to item; shows aggregates are exact sums
/insights	Insights.tsx	Portfolio summary and products rising or falling fastest
/accuracy	Accuracy.tsx	Measured performance: windows, horizon, regimes, occurrence
/validation	Validation.tsx	Replay a historical window where the outcome is known
/model	Model.tsx	Architecture, specification, live re-run of the frozen model
/methodology	Methodology.tsx	Data, limits, capability matrix, what was rejected
/assistant	Assistant.tsx	The AI Forecast Assistant

Components and structure

```
01_PROJECT/frontend/src/
■■■ pages/           the 9 screens
■■■ components/
■   ■■■ layout/      AppShell.tsx, FloatingAIAssistant.tsx
■   ■■■ charts/      ForecastChart.tsx
■   ■■■ diagrams/    index.tsx
■   ■■■ ui/          index.tsx (Card, Badge, Caveat, Explain, Spinner...)
■■■ api/             client.ts, hooks.ts, types.ts
■■■ lib/             format.ts
■■■ test/            5 test files
```

AppShell.tsx is the frame every page sits inside: the header with live RMSE/MAE and API status, the sidebar navigation, and the content area. Pages do not repeat any of that.

ui/index.tsx holds the shared building blocks. Two are worth naming because they carry meaning, not just style:

- Caveat — a warning box used when a number needs a health warning
- Explain — an explanatory box for beginner-facing context

Having them as components means an important caveat looks identical everywhere and cannot be quietly dropped.

4. Routing and navigation

React Router maps a URL to a page. AppShell renders the sidebar and an `<Outlet />` where the current page appears.

Lazy loading. Only overview loads eagerly; every other page is fetched the first time you visit it.

Why? The charting library is 416 kB. If every page loaded up front, the first visit would pay for charts it may never show. Verified build output: Assistant is its own 13.07 kB chunk, downloaded only when you open `/assistant`.

Two ways to reach the assistant, deliberately both:

	Purpose
Sidebar entry "AI Assistant" with its description	tells a first-time visitor the feature exists
Floating circular AI button, bottom-right	a shortcut from any page, for someone already deep in a chart

`FloatingAIAssistant.tsx` is rendered once from AppShell, so every page gets it without opting in. It reveals an "AI Assistant" label on hover **and on keyboard focus**, carries `aria-label="Open AI Assistant"`, marks itself as the current page on `/assistant`, and navigates through the router rather than reloading.

Accessibility. `aria-label` is text read aloud by screen readers when the visible content ("AI") would not make sense on its own. Keyboard focus matters because not everyone uses a mouse.

5. How the frontend talks to the backend

```
Component
  | useForecast('CA_3','FOODS_3_090')
  v
api/hooks.ts      TanStack Query: caching, loading, errors
  |
  v
api/client.ts     builds the URL, parses JSON, maps errors
  |
  v
/api/v1/...       same origin -> nginx proxies to FastAPI
```

No API host is compiled into the bundle. `VITE_API_BASE_URL` is deliberately left unset, so the client falls back to the same-origin path `/api/v1` and nginx proxies it. A test asserts the base URL never matches `localhost`, an IP address or an absolute URL — because a hard-coded host is the classic reason a build works locally and dies in deployment.

`api/types.ts` mirrors the backend's OpenAPI contract in TypeScript. If the backend renames a field, the frontend fails to compile — loudly, at build time, instead of silently showing an empty chart.

6. Loading, error and empty states

Every data-driven view handles three states, not one:

State	What the user sees
Loading	a Spinner with a label saying what is being fetched
Error	an error box with the message and, where available, a remedy
Success	the data

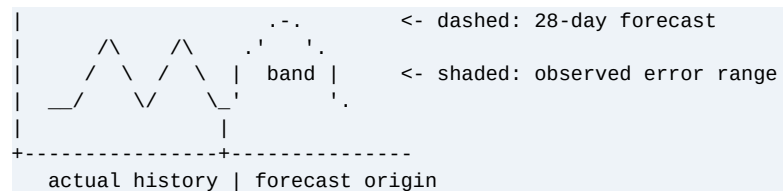
***The rule we hold to:** a failed request must produce a visible error, never a plausible-looking placeholder. A chart showing zeros because the request failed is worse than no chart — it looks like real data.*

There is a test for exactly this: it makes the API fail and asserts an error state appears rather than a rendered-looking empty chart.

A real example. The Assistant page originally had no branch for "the status request failed". When an old API returned 404 for `/genai/status`, every section was gated on data that never arrived and **the page rendered the heading and nothing else**. It now has an explicit error branch naming the failing endpoint and what to do, and a test asserts the page can never be blank again.

7. Charts and visualisation

ForecastChart.tsx (Recharts) draws the one visual that carries the product:



- **solid line** = actual sales
- **dashed line** = the frozen forecast
- **shaded band** = observed past error, not a confidence interval
- **vertical marker** = the forecast origin

The chart's caption says in words that the band is measured model error. That wording comes from the backend's `band_basis` field — the frontend does not invent it.

8. Production build and serving

```
npm run build
  |
  v
dist/  index.html + hashed JS/CSS chunks
  |
  v
nginx container serves dist/ and proxies /api/ to the API
```

VERIFIED build output:

Chunk	Raw	Gzipped
charts	416.32 kB	112.23 kB
react	157.89 kB	51.84 kB
query	47.32 kB	14.73 kB
index	32.45 kB	10.35 kB
Assistant (lazy)	13.07 kB	4.46 kB
CSS	23.55 kB	5.31 kB

SPA deep links. If a user opens `/forecast` directly, the server has no file at that path. nginx is configured with `try_files $uri $uri/ /index.html`, so any unknown path returns the app and React Router shows the right page. Without this, refreshing on `/forecast` would 404.

Caching. Hashed asset filenames are cached for a year; `index.html` is never cached, so a new deploy is picked up immediately.

9. The Assistant page

The page that needed the most care, because it displays AI output.

What it shows: a focus banner when a store-item is selected, starter suggestions from the backend, the conversation, an input, and two explanatory panels — **How this assistant works** and **What it will not do**.

The **provenance strip** under every answer is the important part:

- **which data family** the answer used (`intent`)
- **how long** it took (`elapsed_ms`)
- **whether every figure traced back** to backend data (`grounded`)

When a number cannot be traced, the UI **names it and marks it unverified** rather than hiding it. When the backend refused a request locally, the strip says **"answered locally — no AI call"** with the policy category.

The frontend contains no Gemini key, no AI SDK and no Gemini endpoint. Questions go to our own backend. A test asserts the rendered DOM never contains anything key-shaped, and the built bundle is scanned too.

10. Design rules we enforce in the UI

These are product decisions, not styling:

No fabricated numbers. Every figure on screen came from an API response. Nothing is computed in the browser to look fuller.

No fake confidence intervals. The band is labelled observed error, because the model emits point forecasts only.

No price what-if slider. The research measured the frozen model's response to simulated price changes and found it non-monotone and sometimes economically backwards. A slider would have been the most impressive-looking feature in the app; shipping it would have been misleading, so it does not exist and *methodology* says why.

Validation accuracy is never shown as live accuracy. The delivered forecast window has no ground truth, so no accuracy is quoted against it.

Visual language: deep navy-slate, never pure black, so cards separate from the page and charts stay readable. Restrained borders and shadows — it should read as an analytical instrument, not a consumer app.

11. Testing and verification

VERIFIED: 62 tests across 5 files.

File	Covers
pages.test.tsx	page rendering against a mocked API; navigation; header status
assistant.test.tsx	availability, grounding warnings, refusals, no key in the DOM
floating-assistant.test.tsx	the button exists, is labelled, navigates, coexists with the sidebar entry
client.test.ts	URL building, error mapping, network failure
format.test.ts	number and label formatting

Plus **TypeScript compiles clean** and the **production build succeeds** (~2 s).

The load-bearing assertions are the honesty ones: that the app renders exactly the numbers the API returned, that a failed request produces an error state, that backend caveats reach the screen, and that an AI answer with an untraceable number is flagged rather than shown as fact.

12. How to present this

"I built the frontend — the React app in the browser.

React works by building the interface out of components: small reusable pieces like a card or a badge. We have 9 pages sharing one shell that holds the header and sidebar. We use TypeScript, so the API response shapes are typed — if the backend renames a field, we get a compile error instead of a blank chart.

The frontend holds no data. It asks our FastAPI backend over HTTP and gets JSON back. Importantly, no API address is compiled into the bundle — the app uses a same-origin path and nginx proxies it, so the same build works anywhere.

The main screen is the forecast page: actual sales as a solid line flowing into the 28-day forecast as a dashed line, with a shaded band. The band is **observed past error**, not a confidence interval, and the caption says so — that wording comes from the backend, we don't invent it.

There's also the AI assistant, reachable from the sidebar and from a floating button. Every answer shows which data it used and whether every number traced back to our backend. If a number can't be traced, we name it and mark it unverified rather than hiding it.

Two things we deliberately did **not** build: a price what-if slider, because we measured the model's price response and it's backwards; and any display of accuracy for the delivered forecast, because that window has no ground truth.

62 tests, TypeScript clean, production build verified."

13. Likely judge questions

Beginner

Q: What is React? A: A JavaScript library that builds the interface out of reusable components and updates only the parts that change.

Q: How does the frontend get the data? A: It sends HTTP requests to our FastAPI backend and receives JSON, then draws it.

Q: How many pages? A: 9, plus a floating AI button available on all of them.

Technical

Q: Why TypeScript? A: The API response shapes are typed from the backend's OpenAPI contract, so a contract change is a compile error rather than a silently empty chart.

Q: How do deep links work in a single-page app? A: nginx uses `try_files $uri $uri/ /index.html`, so any unknown path returns the app and React Router picks the page. Without it, refreshing `/forecast` would 404.

Q: Is the API host baked into the build? A: No — deliberately. `VITE_API_BASE_URL` is unset so the client uses the same-origin `/api/v1` path, and a test asserts it never matches localhost, an IP or an absolute URL.

Q: Could the Gemini key end up in the bundle? A: No. The frontend has no key, no AI SDK and no Gemini endpoint — it only talks to our backend. A test asserts the DOM contains nothing key-shaped, and we scan the built bundle.

Q: What happens if the API is down? A: Every view has an explicit error state showing the message and a remedy. We never render a plausible-looking empty chart — that's tested.

Q: Why is the chart band not a confidence interval? A: Because the model emits point forecasts only. The band is measured error from 8 backtest windows, and the caption text comes from the backend's `band_basis` field.

14. Glossary

Term	Meaning
React	Library for building UIs from components
Component	A reusable piece of interface
Props	Inputs passed into a component
State	Data a component remembers between redraws
Hook	A use... function giving a component React features
JSX	HTML-looking syntax inside JavaScript
SPA	Single Page Application — JS swaps content instead of reloading
Routing	Mapping a URL to a page
Lazy loading	Fetching a page's code only when it is first opened
Bundle / chunk	The built JavaScript file(s) the browser downloads
Vite	Our build tool and dev server
TanStack Query	Handles fetching, caching, loading and error state
nginx	The web server that serves the built files and proxies /api/
aria-label	Text read by screen readers when the visible label isn't enough
Same-origin	Frontend and API on the same host, so no CORS is needed